本章路線

這章要回答的核心問題是：如何用可強制執行的資料結構，限制主體對物件的能力，並在撤銷後維持可證明的一致性？

前置知識。 mode bit、access control 基礎、graph 與 matrix。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 區分 subject/object/domain/right
- 比較 ACL 與 capability
- 分析 revocation 對已開 handle 的影響

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.1 Goals of protection

- 政策決定允許什麼，機制保證不可繞過。
- 完整仲介可以核發受保護handle，撤銷語義必須一起定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

保護的問題是「誰可對哪個物件做什麼」

subject/domain 是請求來源，object 是資源，right 是操作。保護機制必須表示並強制執行允許關係；安全（security）還要處理認證、攻擊、故障、稽核與信任。把檔案隱藏在難猜路徑不是存取控制。

政策例：「助教可讀作業，只有任課教師可改成績」；機制例：ACL、capability、受保護的系統呼叫。政策換人/換課時應能改資料，而不是重寫內核。完整仲介（complete mediation）要求每次使用都受有效授權約束；可在 open 時檢查 ACL，再於每次 read/write 檢查已核發 handle 的 mode，但這會影響之後撤銷的語義。

信任邊界不能只靠「呼叫者說自己有權」：reference monitor 必須不可被繞過、保護其自身狀態，並使規則足夠小而可檢查。硬體 mode/address-space 隔離是部分支撐，不自動等於應用政策已正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.2 Principles of protection

- 最小權限限制物件範圍、操作與持續時間。
- 小型授權服務仍須驗物件範圍與caller輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小權限同時限制範圍與時間

備份工作只需讀 `/course/submissions`、寫備份目標，不應同時有修改成績或載入核心模組的權限。若整個程式一直用全權帳號，解析一個惡意檔案的錯誤就可能取得所有權限。拆成低權解析器與小型授權服務，服務仍須驗參數與物件範圍。

權限集合 R(t) 要包含當下必要權限 N(t)，並盡量縮小 R(t)−N(t)。這不是單純「權限數量越少越好」：只有 read 但可讀所有學生個資，仍可能比可寫單一暫存檔風險大。細粒度 scope、持續時間與可委派性都影響損害界。

fail-safe defaults：未找到明確授權就拒絕。稽核記錄 subject、object、operation、decision、時間與關聯 id；有 log 不代表攻擊被阻止。separation of duty 可要求兩角色批准成績發布，但若同一使用者能自行同時啟用兩角色，政策仍可能失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.3 Domain of protection

- Domain、process、user與CPU mode並不一一對應。
- UNIX有效UID切換與Multics gate各限制不同的進入條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Domain 是權限集合，不等於 CPU mode

D={⟨F1,{read}⟩,⟨F2,{read,write}⟩}。同一 user 的兩個 process 可以有不同 domain；同一 process 的受保護呼叫也可能切 domain。user/kernel 兩個 mode 無法單獨分隔所有同為 user mode 的行程，還要有位址空間與個別授權。

靜態 domain 把各階段需求聯集：compile 需讀source、link需寫binary，若全程持兩權就擴大每階段暴露。動態 domain 可在狀態切換時改權限，但切換本身也必須授權、保存/還原正確 caller context，並檢查參數。

傳統 UNIX setuid 執行檔可在成功 exec 的指定條件下改 effective UID 為檔案 owner；不是把 real UID 一律改掉，也不是單純呼叫一般函式就改 UID。例 realUID=1001、owner=0：執行受允許的 setuid 程式後 euid 可為0，realUID仍1001；程式需正確降權、驗證環境與輸入。具體抑制條件依 OS，這裡不把歷史機制當成任意提權方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MULTICS rings：受控進入，不是內圈任意入口

書中歷史抽象模型：ring0較高權；access bracket (b1,b2)，gate limit b3≥b2；caller ring i 在 [b1,b2] 可直接執行。i>b2 時，只有 i≤b3 且入口是指定 gate 才可轉入較高權；i<b1 轉往較低權時，參數不得讓低權程式任意引用其不可見區域。

例 bracket=[1,3]、limit=5、gate={entryA}：i=2 呼叫一般入口可保持ring2；i=4 呼叫entryA可受控進入ring3，呼叫entryB拒絕；i=6即使entryA也拒絕。返回要恢復 caller ring/stack。此例限於書中 ring model；完整 MULTICS 還有 read/write brackets 等硬體細節。

純巢狀權限無法表達互不包含的需求：D甲只需F、D乙只需G，若強制D甲⊆D乙或反之，至少一方會多得不需的物件。一般 access matrix 能表達這種非階層分隔。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.4 Access matrix

- 矩陣以domain為row、object為column，變更操作也受授權。
- copy/owner改object column，control只可刪domain row。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一 access matrix 的初值

授權矩陣與 domain-switch 可達性

domain	F	G	P	D2	D3
D1	r*	ø	ø	switch	ø
D2	w	r	ø	ø	switch
D3	ø	ø	print	ø	ø



switch 可達：D1 → D2 → D3

目前 row 不會自動聯集

r* = read + 同物件授權傳播；ø = 沒有權利。右圖只表示 switch，不代表直接入口任意可用。

令 A[D,O] 是操作集合；空格就是∅，不表示「尚未決定」。目前process只使用當前domain那一row。D1可讀F，不能寫F；D2可寫F且可切D3；D3可印P。D1能切D2再切D3，代表可能取得後續domain的行為能力，不能只看D1原始row就斷言它永遠無法印P。

domain-switch 本身當作對 domain object 的 switch 操作。切換後不是把兩row聯集；若需要受控服務，還應限制入口與參數，避免任意直接進入高權程式中段。

copy、transfer、limited copy、owner、control

模型中的 r* 表示有r且可在**同一object column**傳播r；不能用F的r*給G加r。普通copy保留來源；transfer移除來源再給目標；limited copy給r但不給r*。三種變更不可混用。

初始 A[D1,F]={r*,owner}、A[D2,F]={w}、A[D3,F]=∅：D1 limited-copy給D3後，D3={r}，不能再轉授。D1作為owner可在F整個column增刪指定權利，如刪D2的w；owner(F)不等於owner所有物件。

control 只對domain object：若 control∈A[D1,D2]，本模型允許D1從D2的row移除權利，不是任意新增。若刪D2對D3的switch，原來D1→D2→D3的可達路徑也中斷；需重新分析權限變更後的可達性。

保護權利不等於禁止資訊流

即使D3只有read(F)且沒有copy(F)權利，只要它能read資料並write(G)，就可把內容逐byte寫到G。copyright限制的是授權的傳播，不是資訊的複製。已讀進user記憶體的資料通常也不能透過撤銷F的read「收回」。

可用不變量證明受限系統安全：若唯一授權操作是同column、只能授出來源已有權利的attenuation，且沒有owner/建立新權利規則，則任一後繼狀態的權利都源自初始某個domain對同object持有的權利。歸納：初始成立；每一步新增r必有來源r，沿來源鏈回溯到初始。此結論不保證機密資訊不外流，也不能套到允許owner任意造新權利的模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.5 Implementation of access matrix

- ACL按column，capability按row；需保留相同權限語義。
- cached open handle不必隨ACL刪除自動失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏矩陣的四種實作

表示	索引方向	回答什麼最快	更新/保護責任
global triples (D,O,R)	全域或雙索引	任一pair	保護全域表，避免每次線性掃全表
ACL(O)=[(D,R)]	object column	誰可用O	可集中刪O的授權；須定義matching/default規則
capability list(D)	domain row	D目前持有哪些能力	cap不可偽造；傳遞/attenuation由機制控制
lock/key	object locks與domain keys	是否有matching key	key/lock清單本身受保護，換key可撤銷

capability 是受保護的物件參考＋權利；數字fd是本行程表的索引，不能靠猜別人的fd就取得其capability。不可偽造可由kernel table、tagged hardware或適當的受保護表示達成；不是「所有capability都必須永遠不能出現在user位址空間」。

同一權限的 row／column 轉換與檢查

F的ACL=[D1:r,D2:rw]，G的ACL=[D2:r,D3:w]。轉成cap lists：D1=[F:r]；D2=[F:rw,G:r]；D3=[G:w]。兩者在同一時點表示相同矩陣，但委派/撤銷的更新成本不同。若採hash索引查pair可期望O(1)，線性list查長k則O(k)，不能把表示名稱直接當作固定O(1)。

hybrid：open時檢ACL，給kernel保存的handle{F,read};後續write查mode拒絕。若此後ACL刪read，舊handle是否仍可read依撤銷政策；「改ACL必立刻撤所有已開啟fd」並非UNIX的一般保證。新增每次generation檢查可改此語義，但需處理mapping與in-flight操作。

教學ACL採只有allow且union/default deny；真系統若有deny ACE/順序/群組規則，需要另一套明確求值演算法，不能把此簡化算法套到Windows DACL。

14.6 Access control

- 歷史Solaris10案例與教學RBAC模型分開。
- active roles不能超出assigned roles，互斥政策要明定session或user範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RBAC：user→role→權限與任務的映射

歷史案例固定為 Solaris 10。其RBAC把roles、rights profiles、authorizations與process privileges分開：role聚合管理任務，profile可指定命令與執行屬性，authorization可由特定應用檢查，privilege由kernel對操作檢查。不是所有角色權利都只是某個UID，也不是authorization字串本身直接替代kernel privilege。

原創模型：user U可assume BackupRole；該role可執行BackupCmd，執行時只帶所需讀取/目標寫入權；不能改GradeDB。user V可assume GradeRole，但不能替自己核准發布。啟用角色的認證與稽核要另做，單看user-to-role配置不能推出所有請求均允許。

形式化簡化RBAC：UA⊆Users×Roles、PA⊆Roles×Permissions、session activeRoles⊆assignedRoles(U)，當且僅當某active role含所需permission才允許。若有互斥roles，啟用時再驗constraints。此式是教學RBAC模型，不等同Solaris全部privilege集合運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.7 Revocation of access rights

- 定義撤銷邊界，避免cached handle與slot重用漏洞。
- 撤銷後slot重用必區分object版本並同步resolve。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

撤銷有四個獨立維度

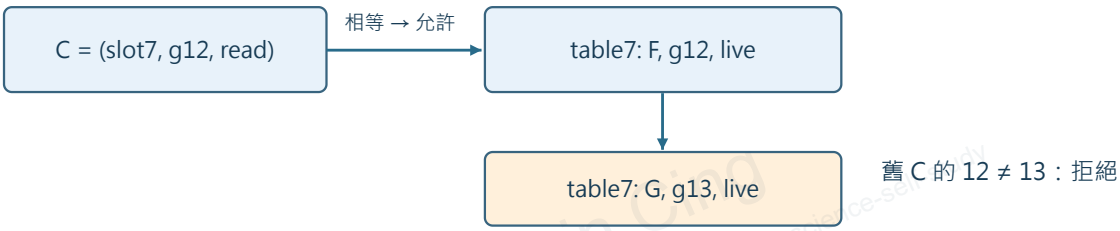
immediate/delayed：何時生效；selective/general：影響哪些持有人；partial/total：撤哪些操作；temporary/permanent：未來是否可再授權。即刻撤銷的精確邊界通常是「之後新操作不得通過check」，不能假定已開始的I/O會自動取消，或已讀內容會自動消失。

ACL集中刪entry容易，但hybrid既存handle/cache可使效果延後。capability分散時可採reacquisition/租期、back-pointers、indirection、key/version。reacquisition給最長延遲界；back-pointers需追蹤所有複本；indirection把查驗收斂到一個可撤銷slot；master key換值會使共用同key的能力全失效，若要selective需不同代理/分組key。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Indirection 撤銷：generation 避免 ABA

同一 slot 重用：不能只比較索引 7



撤銷 / 重用時換 generation；resolve 的檢查與取得物件 reference 必須同步。

cap C={slot=7,generation=12,rights=r}；table[7]={obj=F,g=12,live=true}。每次resolve先驗live與generation相等，再查r，並在同一同步保護下取得obj存活reference。撤銷設live=false；重用slot存G時generation升13。舊C仍是(7,12)，對(7,13)拒絕，不能錯指G。

Indirection 撤銷：generation 避免 ABA （續）

一般證明條件： 不重用仍可能存在的generation，且check→取得穩定reference不可被撤銷/重配插入。則接受C必對應它核發時的物件版本。若generation可wrap且舊C永存，終會ABA；必須不重用或先確保舊tokens全消失。若只驗slot數字，F撤銷後G占slot7就會讓舊cap取得G，是確定的權限漏洞。

14.8 Capability-based systems

- Hydra/CAP以受保護typed procedure支援抽象操作。
- amplification限型別與procedure，不能由caller任意要求升權。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hydra：抽象操作與受限 amplification

歷史Hydra讓user-defined type有auxiliary rights。原創例 Account物件A的client cap只含deposit，不含代表資料的raw read/write。呼叫受信任deposit(A,10)時，機制確認型別與操作權，受保護procedure暫時獲得實作A所需的representation access，更新balance=40→50；返回後client仍只有deposit。

amplification不是「caller自選要升權」；只由已認可的procedure、指定型別與受限規則觸發。若任意routine都能把readonly參數升成write，就破壞caller的保護需求。因此互不信任subsystems需要控制可放大的權利、可保存的參考及參數範圍；不能只說封裝就自動安全。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cambridge CAP：資料能力與軟體能力

data capability由machine/microcode強制read/write/execute段權限；software capability的自定義意義由受保護procedure解釋，底層保證不能任意偽造/篡改。seal/unseal式權限讓只有對應型別與環境的procedure能檢視受保護表示。

與Hydra共同點：client只拿抽象操作能力，implementation可能需要更低層存取；邊界是受保護的typed procedure。差別是權利解釋與支援庫的設計，不是「CAP能執行任何user程式就一定安全」。若Bank子系統的procedure錯把balance寫壞，Bank語義仍壞；隔離只把損害限制在其保護環境，不能證明Bank業務不變量。

本節按原書歷史架構比較，Account數值為原創抽象模型，不冒充Hydra/CAP可直接執行的原始API。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.9 Language-based protection

- 語言機制需要runtime/loader/硬體信任，歷史Java以stack inspection示範。
- privileged frame先驗自身權利；掃完普通frames還須驗inherited context。

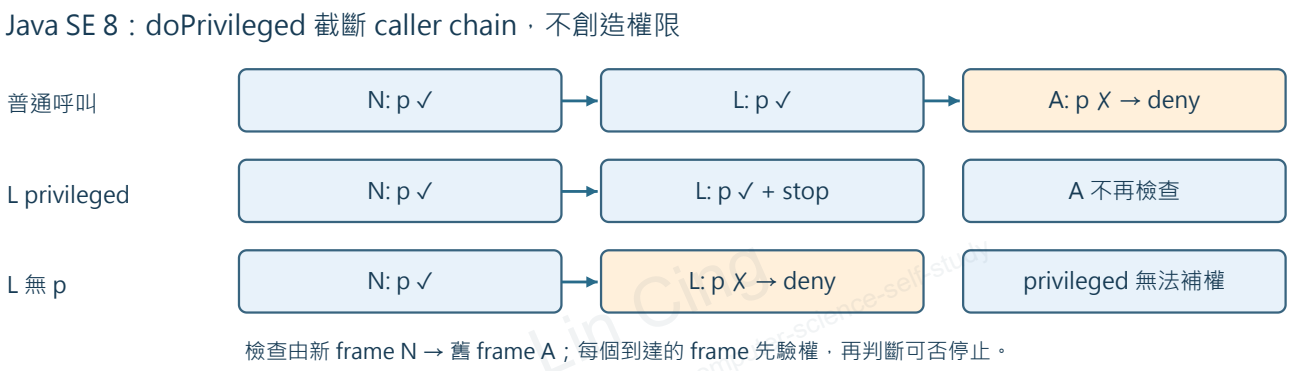
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

編譯期、load-time、run-time 各能保證什麼

language把存取宣告放在type/module/interface邊界；compiler可靜態排除非法操作，runtime檢查動態index/type/capability。若執行者可改機器碼、偽造reference或透過不受管native入口繞過檢查，單靠原始碼private欄位不足以隔離敵對程式。

type safety保證類型/記憶體規則，不代表業務授權。能合法取得Account reference的程式，仍可能錯誤地允許任意提款；type-correct也可能資訊外洩。kernel隔離粗到process，language可細到同process內的class/object，但其runtime、loader、native interface進入trusted computing base。

Java SE 8 的 stack inspection：privileged 也先驗權



限於Java SE 8傳統AccessController模型、無explicit context/limited permissions的doPrivileged。本例call stack（新→舊）Networking N→Loader L→Applet A；目標permission p=connect(proxy.example:80)，N與L有p，A沒有。普通呼叫逐frame查，N通過、L通過、A拒絕；L若在自己的有權domain下使用doPrivileged，N通過、L先驗p再停，A不再列入這次檢查。

Java SE 8 的 stack inspection：privileged 也先驗權 （續）

若L本身沒有p，標privileged也會在L拒絕；它不是授予新權限。所有顯式frames通過後還須檢查inherited access-control context，不是「掃完stack就依實作隨意准駁」。此處不把歷史SecurityManager沙箱模型宣稱為現代Java建議方案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Confused deputy：在高權框架驗證實際目標

Loader為了讓低權Applet讀取代理服務，可在受控privileged區塊連到固定proxy。若Loader把Applet給的任意addr原封不動放入doPrivileged，會讓低權caller借Loader的網路權掃內網。正確例先把目的地固定在已驗證的proxy、限制操作與回傳資料，不把整個user callback都納入高權區塊。

stack inspection的歸納條件：每個已檢查的frame都具有p；遇到privileged邊界時，其本身也具有p才停止。由此可保證「本次授權不超過邊界caller已有的p」，但仍須信任邊界caller選擇了正確object/參數，不能據此證明整體服務政策正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從矩陣到實作的等價條件

access matrix只指定當前允許關係；ACL/capabilities是不同索引，dynamic operations再改關係。實作要保存object identity、right的型別與domain context，並確保不可偽造、不可繞過。若查驗結果被cache，必須把撤銷語義一起定義。

owner操作column，control操作domain row的刪除；switch改目前domain；copy/transfer/limited-copy依規則改同object授權。這些操作的組合會改未來可達domain，所以單看靜態row不能推出整段程式的權限上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自學時可檢驗的三類結論

①授權安全：每次操作是否有相應權利；②狀態安全：撤銷、重用、競爭後是否錯指物件；③資訊/業務安全：即使所有權限檢查都通過，是否仍可藉合法read/write外洩或濫用高權服務。三者需要不同證據。

一般證明必須先限定允許的狀態轉移。cap attenuation的歸納不適用於有任意owner加權的系統；generation證明不適用於無界舊token搭配有限wrap；Java privileged邊界也不免除參數驗證。練習以完整矩陣、slot版本與call stack把這些界線具體化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Silberschatz、Galvin、Gagne，*Operating System Concepts*，第九版，第14章，印刷頁625–650。原創矩陣、授權例、generation模型與練習依書中議題重建，未重製書中圖題。

Oracle Java SE 8 AccessController API：核對先驗frame權限、privileged停止條件及inherited context；教材限定歷史SE8模型。

Solaris 10 System Administration Guide: RBAC：核對roles、profiles、authorizations、privileges的分工；未用Solaris11替代歷史案例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study